

Venue Booking Platform

A SaaS booking system for badminton, futsal & padel venues in Indonesia

Prepared by: Project Lead

Audience: Co-founder / Development Partner

Purpose: Shared understanding of what we're building, who pays for it, and how the system works end-to-end before development starts

Status: Pre-development — planning & architecture phase

Revision: v1.2 — added Section 3.1 (User Roles & Access); clarified Super-Admin is internal-only and out of MVP scope

1. What We're Building

We're building a **multi-tenant SaaS platform** for sports venue booking, targeting small and mid-size venue owners in Indonesia — badminton courts, futsal fields, and padel courts. The platform replaces the manual, WhatsApp-driven booking process most venue owners rely on today with a proper booking system, online payments, and a customer-facing marketplace.

The important word is "**SaaS**": venue owners don't buy software from us once — they use our service continuously, and we earn money every month and on every transaction. This shapes almost every architectural decision we make, especially around multi-tenancy (covered in Section 3).

2. Business Model — How We Make Money

We earn revenue from **two separate layers**, from two different kinds of users:

LAYER 1 — VENUE OWNERS

Subscription fee (monthly/annual), charged to the venue owner for using our platform to manage their venue, bookings, and dashboard. This is our recurring, predictable revenue.

LAYER 2 — CUSTOMERS

Transaction fee, a small percentage taken from each booking payment made by a customer through the platform. This scales with platform usage/volume.

Why this matters for dev: our billing logic needs to track two independent money flows — (1) what the venue owner owes us for the subscription, and (2) what we deduct from each customer transaction. These should be modeled as separate ledgers, not bundled into one "payment" concept.

3. Platform Architecture — Multi-Tenancy

Because venue owners only use *our* service (they don't run their own separate app), every venue, court, booking, and review record in the database must be scoped to a specific owner. This is called **multi-tenancy**, and it's the single most important architectural decision to lock down before writing any code — changing it later means touching almost every table and query.

What this means in practice

- Every `Venue` record has an `ownerId` — owners only ever see and manage their own venues
- Customers are **not** tied to any single venue — they browse and book across *all* venues on the platform, like a marketplace (similar to how Gojek/Grab work with merchants)
- Admin dashboards are scoped per owner: an owner's dashboard shows only their own bookings, revenue, and reviews — never another owner's data

3.1 — User Roles & Access NEW

Multi-tenancy isn't just a data-modeling rule — it has to map to a concrete role system, since "who can see/do what" is what actually enforces tenant isolation. The platform has **two MVP-facing roles**, plus one internal role that exists but is deliberately out of MVP scope:

Role	Scope	Can do	MVP?
Customer	<i>Global (cross-tenant)</i>	Browse all venues across all owners, book a slot, pay, view their own booking history, leave reviews after a completed session	Yes
Venue Owner (Admin)	<i>Scoped to own <code>ownerId</code></i>	Manage their own venues/courts, view only their own bookings/revenue/reviews, choose subscription plan, receive payouts	Yes

Role	Scope	Can do	MVP?
Platform Super-Admin	<i>Global, cross-tenant</i>	Internal team only — manage owner accounts, handle support/disputes, view platform-wide metrics, moderate listings. Not customer or owner facing.	No — internal tooling, see note below

Enforcement note for dev: role scoping must happen at the query layer (e.g. every owner-facing query implicitly filters by `WHERE ownerId = currentUser.id`), not just by hiding UI elements. Customer and Owner share the same login system but get fundamentally different data access.

Why Super-Admin has no workflow or MVP feature: per the golden rule in Section 4, MVP only includes what's required for one full booking to happen — and a booking doesn't need a platform-level admin. Super-Admin is a real role we'll need eventually (support, disputes, moderation), but it gets *no dedicated UI or workflow in MVP*. Until then, the founding team can operate as Super-Admin via direct database access or a minimal internal script. Revisit this once we have enough owners that ad hoc access stops being safe or practical — likely a V2/V3 item, not before.

Decision needed before MVP starts: do venue owners get their own branded sub-page (e.g. *yourapp.com/venue-name*), or does everything live under one shared marketplace homepage where customers browse all venues together? This changes routing, branding, and parts of the data model — decide together before coding begins.

4. Core Workflows

There are two distinct user journeys running on top of the same platform. (*Super-Admin has no workflow here by design — see 3.1.*)

4.1 — Venue Owner Workflow (our paying customer)

- 1 **Sign up** — owner creates an account and registers their business
- 2 **Set up venue(s)** — add venue(s) and courts/fields, set operating hours, price per hour, photos
- 3 **Choose a plan** — free tier or paid subscription tier (this is where our recurring revenue starts)
- 4 **Manage via dashboard** — owner sees only their own bookings, revenue, and customer reviews
- 5 **Receive payouts** — money from customer bookings (minus our transaction fee) flows to the owner

4.2 — Customer Workflow (the people booking courts)

- 1 **Browse / search** — customer searches venues by category (badminton/futsal/padel), price, location
- 2 **Pick a venue & time slot** — system shows only real, non-overlapping available slots
- 3 **Book & pay** — booking is created as `PENDING`, payment is processed through our gateway
- 4 **Booking confirmed** — status moves to `CONFIRMED` once payment succeeds
- 5 **Play, then review** — after the session, status becomes `COMPLETED` and the customer can leave a rating

Golden rule for what's "core": if removing a piece of the workflow breaks one full booking from happening, it belongs in the MVP. If it just makes the experience nicer, it belongs in V2 or V3.

5. Feature Roadmap

The full feature list is split into three delivery phases. We build and ship MVP first, completely, before touching V2 or V3.

#	Feature	Phase	What it does
1	Venue Management	MVP	Owners add/edit/deactivate venues and courts, grouped by venue type
2	Booking + Status Lifecycle	MVP	PENDING → CONFIRMED → COMPLETED , or PENDING → CANCELLED
3	Availability Checker	MVP	Overlap-detection logic prevents double-booking the same court/time
4	Dashboards (User + Admin)	MVP	Basic visibility into bookings, totals, and revenue. "Admin" here means Venue Owner admin (Section 3.1) — Super-Admin tooling is out of MVP.
5	Time Slot System	V2	Fixed, generated bookable slots instead of free-typed time ranges
6	Venue Gallery	V2	Multiple photos per venue/court
7	Review & Rating	V2	Customers rate/review venues after a completed booking
8	Search & Filter	V2	Filter by category, price, rating, location
9	Real Payment Integration	V3	Live gateway (Midtrans/Xendit) replacing simulated payments
10	Promo System	V3	Discount codes with expiry, applied at checkout
11	Membership Tiers	V3	Bronze/Silver/Gold tiers with discounts and booking perks
12	Peak Hour Pricing	V3	Dynamic pricing based on time of day

6. Payment Integration

Since we take a transaction fee on every customer booking, we need a real payment gateway — not just a simulated one — once we move past MVP. The two realistic options for Indonesia:

- **Midtrans** — well-established, widely used by Indonesian platforms, supports QRIS, e-wallets (GoPay/OVO/DANA), virtual accounts, and cards
- **Xendit** — similar coverage, often considered slightly simpler to integrate for a first-time gateway setup

QRIS and e-wallet support are **non-negotiable** — most customers booking a casual badminton or futsal session pay this way, not by card. Our `Payment` entity needs a status flow of its own: `PENDING` → `PAID` / `FAILED` / `REFUNDED` , separate from the booking status.

7. AI Integration

We have budget allocated for AI features. These are scoped as differentiators, not MVP requirements — they come in after the core platform works.

Feature	Why it's useful
WhatsApp-style chatbot	

Feature	Why it's useful
	Handles "is Court A free tomorrow at 8pm?" type questions automatically — reduces the WhatsApp load venue owners currently deal with manually. Likely our strongest selling point to owners.
No-show prediction	Flags bookings likely to be cancelled/no-show so owners can manage capacity smarter
Smart scheduling	Suggests off-peak slots to customers, helping balance demand against Peak Hour Pricing (V3)
Review summarization	Auto-summarizes sentiment across reviews instead of owners reading every comment
Dynamic pricing suggestions	Suggests peak/off-peak pricing based on historical booking demand

8. Tech Stack & Team Split

Tech stack — to confirm together before MVP starts: backend framework, frontend framework, and database need to be agreed and locked as a first step. Once decided, list them here so this document stays the single source of truth.

Suggested team split

PERSON A — BACKEND

Venue management, booking engine, availability checker (overlap logic), payment gateway integration, admin/owner dashboard APIs, role & access enforcement

PERSON B — FRONTEND

Customer-facing booking flow, time slot UI, venue gallery/search, reviews, owner and customer dashboard UI

9. Immediate Next Steps

- 1 Agree on the tech stack (backend, frontend, database) — together, once, no revisiting
- 2 Finalize the data model for MVP entities: Venue, Booking, User, Payment — including the `role` field on User (Section 3.1)
- 3 Decide the branded-page vs. single-marketplace question (Section 3)
- 4 Set up shared repo and basic project structure
- 5 Split MVP tasks per the roles above and set an MVP-only deadline
- 6 Payment gateway and AI features come after MVP is live and tested end-to-end

This document reflects current planning as of the project's pre-development phase. Update it as decisions get made — it should always describe what we're actually building, not what we used to plan.